



FLORAai

AI-Powered Plant-Health Diagnostics for
Indian Subcontinent Spice & Medicinal Plants

Complete Project Report, Tech Stack & Development Plan

Authored & owned by: amrutachari

Contact: amrutachari11112086@gmail.com

Stack: React - FastAPI - MongoDB - Gemini Vision AI

1. Project Overview

FLORAai is a full-stack web application that lets growers photograph a plant leaf and instantly receive an AI-generated health report - a health score, the probable disease, a confidence level and an organic remedy plan - tailored to Indian subcontinent spice, medicinal and household plants. Every scan is saved privately to the grower's account so recovery can be tracked over time.

The platform is built around a complete, security-hardened authentication system with role-based access for regular users and administrators. Visitors can browse the marketing homepage freely but are prompted to sign in to scan plants, save reports, view history or manage preferences.

User Personas

- Grower (user): scans plants, tracks health history, manages profile and preferences.
- Administrator: monitors platform stats, manages user roles, reviews scan logs and regional outbreaks.

2. Key Features

Authentication & Accounts

- Email + password registration with strong-password enforcement (frontend + backend).
- Login, logout, silent token refresh, and 'remember me'.
- Email verification and secure password reset (tokenised, time-limited links).
- Profile management, preferences, privacy settings, password change.
- Account deletion (soft delete) and full data export (JSON).
- Role-based access control; protected routes redirect unauthenticated users to login.

Plant Diagnostics

- Upload a photo OR capture live from the device camera on the dashboard.
- AI diagnosis via Google Gemini 3.1 Pro vision, with a deterministic fallback if the model is offline.
- Health score gauge, status badge, confidence bar, detected issues and remedy steps.
- Private, searchable, filterable scan history with detail view, delete and export.

Admin & Platform

- Admin dashboard: total users, admins, scans, verified users; diagnoses-by-status chart.
- Regional outbreak hotspots - severity bars aggregated by grower location.
- User role management (promote/demote) and a full scan-log table.
- Weekly health-digest email to growers, scheduled via platform cron (Mondays 8am IST).
- App-wide light/dark botanical theme, saved to each user's preferences.

3. Supported Plant Species

FLORAAi's knowledge base and AI prompt are tuned for the following 12 species, each mapped to its common diseases/pests and organic remedies:

- Curry Leaf (*Murraya koenigii*)
- Cardamom (*Elettaria cardamomum*)
- Turmeric (*Curcuma longa*)
- Black Pepper (*Piper nigrum*)
- Cinnamon (*Cinnamomum verum*)
- Clove (*Syzygium aromaticum*)
- Tulsi / Holy Basil (*Ocimum sanctum*)
- Neem (*Azadirachta indica*)
- Mango (*Mangifera indica*)
- Aloe Vera (*Aloe barbadensis*)
- Coconut (*Cocos nucifera*)
- Money Plant / Pothos (*Epipremnum aureum*)

4. Technology Stack

Frontend

Framework	React 19 (Create React App + CRACO)
Routing	react-router-dom 7
Styling	Tailwind CSS 3, custom botanical CSS variables
UI kit	shadcn/ui (Radix primitives), lucide-react icons
Theming	next-themes (light/dark class strategy)
Data/HTTP	axios (httpOnly-cookie session), @tanstack/react-query
Charts	Recharts
Feedback	Sonner toasts
Fonts	Playfair Display, Manrope, JetBrains Mono

Backend

Framework	FastAPI (Python)
Database	MongoDB via Motor (async)
Auth	JWT (PyJWT), bcrypt password hashing
AI	Gemini 3.1 Pro vision via emergentintegrations (universal LLM key)
Email	Resend managed integration (httpx) with anti-phishing gate
Scheduling	Platform cron (.emergent/crons.yml) -> webhook endpoint
Server	Uvicorn, managed by Supervisor

5. Architecture

- Single-origin deployment: React frontend and FastAPI backend share one host via ingress; all backend routes are prefixed with /api.
- Stateless JWT access token (15 min) + refresh token (7 days), both stored as httpOnly, Secure, SameSite=None cookies - never exposed to JavaScript or URLs.
- A sessions collection backs refresh tokens so logout, password reset and account deletion revoke them server-side (true token invalidation).
- Plant images are sent as base64 to the backend, analysed by Gemini, and the structured JSON result is persisted per user.
- Backend modules: server.py (routes), emailer.py (templates + safe-send gate), plant_ai.py (vision + fallback).

6. Security Measures

- bcrypt password hashing; strong-password policy (8+ chars, upper, lower, digit, special).
- Short-lived access tokens + refresh rotation; expiry enforced; logout invalidation.
- Brute-force protection: 5 failed logins lock the account for 15 minutes (proxy-IP independent).
- Account-enumeration-safe responses on login and forgot-password.
- Role-based guards on both frontend routes and backend endpoints.
- Per-user data isolation on all scan/history/export queries.
- Pydantic validation at all boundaries; restricted CORS with credentials.
- Cron webhook protected by a constant-time bearer-secret comparison.
- All secrets (JWT, DB, API keys) stored in environment variables, never in code or logs.

7. API Endpoints

Authentication

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/logout
- POST /api/auth/refresh
- GET /api/auth/me
- POST /api/auth/verify-email
- POST /api/auth/resend-verification
- POST /api/auth/forgot-password
- POST /api/auth/reset-password

Users & Scans

- PATCH /api/users/me
- PATCH /api/users/me/password
- DELETE /api/users/me
- GET /api/users/me/history
- GET /api/users/me/export
- GET /api/spices
- POST /api/scans
- GET /api/scans
- GET /api/scans/{id}
- DELETE /api/scans/{id}

Admin & Automation

- GET /api/admin/stats
- GET /api/admin/users
- GET /api/admin/scans
- GET /api/admin/outbreaks
- PATCH /api/admin/users/{id}/role
- POST /api/cron/weekly-digest (webhook, bearer-secured)

8. Database Collections

users	profile, bcrypt hash, role, preferences, privacy, status, timestamps (email unique index)
sessions	refresh-token jti, user_id, revoked flag, expiry (revocation store)
scans	user-scoped diagnosis: plant, status, score, confidence, issues, remedies, image
password_reset_tokens	token, user_id, expiry (TTL index), used flag
email_verification_tokens	token, user_id, expiry (TTL index), used flag
login_attempts	brute-force counter + lockout window per email
audit_logs	security events: login, register, reset, role change, deletion

cron_runs idempotency keys for scheduled webhook deliveries

9. Step-by-Step Development Plan

Step 1 - Requirements & choices

Gathered auth method (JWT email+password), email provider (Resend), AI model (Gemini vision with mock fallback) and admin seeding.

Step 2 - Design language

Defined the organic/earthy botanical theme: deep-forest emerald, lime pulse and spice-terracotta palette, Playfair Display + Manrope + JetBrains Mono typography.

Step 3 - Backend foundation

Modelled MongoDB collections with indexes, TTL and unique constraints; implemented bcrypt hashing and JWT access/refresh tokens as secure cookies.

Step 4 - Auth endpoints

Built register, login, logout, refresh, me, email verification, forgot/reset password with brute-force protection and enumeration-safe responses.

Step 5 - AI diagnosis

Integrated Gemini 3.1 Pro vision for structured plant-health JSON with a deterministic fallback; persisted per-user scans.

Step 6 - Frontend experience

Created the public homepage, split-screen auth pages, dashboard, history, profile, settings and admin - all responsive and accessible with data-testid hooks.

Step 7 - Guards & isolation

Added ProtectedRoute/AdminRoute, auto token refresh, and strict per-user data scoping.

Step 8 - Testing & fixes

Ran automated backend + UI tests; fixed datetime timezone bugs and the brute-force identifier; verified end-to-end via curl and the browser.

Step 9 - Feature expansion

Added 6 more plant species, live camera capture, app-wide dark mode, the weekly health-digest cron, and the admin regional outbreak map.

Step 10 - Branding & docs

Applied the FLORAai leaf logo across the app, added the ownership footer, and produced this report and the project README.

10. Local Development Setup

- Services run under Supervisor: backend (FastAPI :8001), frontend (React :3000), MongoDB.
- Restart: 'sudo supervisorctl restart backend' / 'restart frontend'.
- Backend env (.env): MONGO_URL, DB_NAME, JWT_SECRET, ADMIN_EMAIL, ADMIN_PASSWORD, EMERGENT_LLM_KEY, EMERGENT_EMAIL_KEY, EMAIL_FROM_NAME, FRONTEND_URL, WEBHOOK_CRON_SECRET.
- Frontend env (.env): REACT_APP_BACKEND_URL.
- A default admin is seeded on startup; password-reset and verification links are also logged to the backend with a [DEV] prefix for local testing.

Credits

FLORAai - designed, built and owned by amrutachari. All rights reserved by amrutachari.